Programador (Orientado a Objetos) [Nivel 2]
Lección 4 / Actividad 1

Aplicación avanzada de la programación orientada a objetos

IMPORTANTE

Para resolver tu actividad, **guárdala** en tu computadora e **imprímela**.

Si lo deseas, puedes conservarla para consultas posteriores ya que te sirve para reforzar tu aprendizaje. No es necesario que la envíes para su revisión.

Propósito de la actividad

Reforzar conceptos avanzados del paradigma orientado a objetos e implementar la sintaxis de Python para reutilizar código y extender funciones creadas.

Practica lo que aprendiste

I. Escribe sobre la línea correspondiente una H si el enunciado se refiere a la Herencia, una E si se trata de Encapsulamiento o una A si es sobre clases Abstractas.

H Permite a una clase usar los métodos y atributos definidos en otras clases.

A Las clases definidas a partir de la estructura declarada en otras clases se les llama implementaciones.

H Las clases definidas a partir de la definición otra clase se denominan subclases.

E Los atributos y métodos que se usan sólo dentro del objeto pueden ser ocultados para evitar su uso en el programa principal.

E Los atributos y métodos se ocultan agregando dos guiones al inicio de su identificador.

A La implementación debe contar con una definición para cada método declarado bajo el decorador @abstractmethod.

A Si en la clase nueva existe una definición nueva para un método de la superclase se dice que el método está sobreescrito.

E Si se intenta acceder a un método o atributo oculto desde el programa principal, éste no lo reconocerá como miembro del objeto.

A No se puede crear una instancia directa de una clase cuya definición comienza con la instrucción "__metaclass__= ABCMeta".

II. Observa el código de Python, anota en la primera línea si se trata de herencia, encapsulamiento o abstracción, y contesta las preguntas.

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre
    def hablar(self, *palabras):
        for frase in palabras:
            print self.nombre, ':', frase
class Deportista(Persona):
    def __init__(self, edad, nombre, deporte):
        self.edad = edad
        self.nombre = nombre
        self.deporte = deporte
    def practicarDeporte(self):
        print self.nombre, ": voy a practicar",
self.deporte
luis = Deportista(18, "Luis","natacion")
luis.hablar("Hola ","Estoy hablando")
luis.practicarDeporte()
```

a) Tipo: Herencia

b) ¿Cuál es la superclase?
Persona

c) ¿Cuál es la subclase?
Deportista

d) ¿Qué método se sobreescribe?
practicarDeporte

```python
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre
    def hablar(self, *palabras):
        for frase in palabras:
            print self.nombre, ':', frase
class Deportista(Persona):
    def __init__(self, edad, nombre, deporte):
        self.edad = edad
        self.nombre = nombre
        self.__deporte = deporte
    def practicarDeporte(self):
        print self.nombre, ": voy a practicar"
    def verMiDeporte(self):
        return self.__deporte;
luis = Deportista(18, "Luis","natación")
luis.hablar("Hola, estoy hablando","Este soy yo")
luis.practicarDeporte()
print "Luis practica", luis.verMiDeporte()
```

a) Tipo: Encapsulacion

b) ¿Qué atributo está oculto?
deporte

c) ¿Qué método se usa para manipular el atributo oculto?

verMideporte

```
from abc import ABCMeta, abstractmethod
class Persona:
    __metaclass__= ABCMeta
    def __init__(self, nombre):
        self.nombre = nombre
    def hablar(self, frase):
        print self.nombre, ': ', frase
    @abstractmethod
    def ejercitar(self): pass
class Deportista(Persona):
    def __init__(self, edad, nombre, deporte):
        self.edad = edad
        self.nombre = nombre
        self.deporte = deporte
    def competir(self):
        print self.nombre, ": voy a comeptir en", self.deporte
    def hablar(self, *frases):
        for frase in frases:
            print self.nombre, ': ', frase
    def ejercitar(self):
        print self.nombre, ": voy a practicar",self.deporte
luis = Deportista(18, "Luis","natacion")
luis.hablar("Hola, estoy hablando","Este soy yo")
luis.competir()
luis.ejercitar()
```

a) Tipo: abstraccion

b) ¿Qué clase es la implementación?

Deportista

c) ¿Qué método es el que se está implementando?

hablar

III. Usa tu IDE para crear dos subclases de la siguiente clase :

```
class Animal:
    def __init__(self):
        print "Ha nacido un animal"

    def rugir(self):
        print "Hace algun ruido"
```

Solucion:

```
class Animal: # Clase base Animal
    def __init__(self): # Constructor de la clase Animal
        print("Ha nacido un animal!") # Mensaje de creacion del animal
    def rugir(self): # Metodo rugir
        print ("Y hace un ruido") # Mensaje de rugido del animal


class Perro(Animal): # Clase hija Perro que hereda de Animal
    def __init__(self): # Constructor de la clase Perro
        super().__init__() # Llama al constructor de la clase padre
        print("Ha nacido un perro") # Mensaje de creacion del perro
    def rugir(self): # Metodo rugir sobreescrito
        print("El perro ladra: ¡Guau guau!")   # Mensaje de ladrido del perro
```

IV. Crea dos subclases de "FiguraRegular" que definan el método para calcular su área respectiva según el número de lados, y almacenarla en el atributo "area". Crea una instancia de cada una.

```
class FiguraRegular:
    def __init__(self,lado = 3):
        self.lado = lado
        self.__area = 0
    def verArea(self):
        return self.__area
```

Solucion:

```
class FiguraRegular(ABC): # Clase base FiguraRegular
    def __init__(self, num_lados): # Constructor de la clase FiguraRegular
        self.num_lados = num_lados # Atributo num_lados
        self.area = 0 # Atributo area inicializado en 0

    @abstractmethod # Metodo abstracto para calcular el area
    def calcular_area(self):
        pass
class Cuadrado(FiguraRegular): # Clase hija Cuadrado que hereda de
FiguraRegular
    def __init__(self, lado): # Constructor de la clase Cuadrado
        super().__init__(4) # Llama al constructor de la clase padre con 4 lados
        self.lado = lado # Atributo lado

    def calcular_area(self): # Metodo para calcular el area del cuadrado
        self.area = self.lado ** 2 # Calcula el area del cuadrado
        return self.area # Retorna el area calculada
class Triangulo(FiguraRegular): # Clase hija Triangulo que hereda de FiguraRegular
    def __init__(self, base, altura): # Constructor de la clase Triangulo
        super().__init__(3) # Llama al constructor de la clase padre con 3 lados
        self.base = base # Atributo base
        self.altura = altura # Atributo altura

    def calcular_area(self): # Metodo para calcular el area del triangulo
        self.area = (self.base * self.altura) / 2 # Calcula el area del triangulo
        return self.area # Retorna el area calculada
# Crear instancias y calcular areas
cuadrado = Cuadrado(5) # Instancia de Cuadrado con lado 5
print("Área del cuadrado:", cuadrado.calcular_area()) # Imprime el area del
cuadrado
triangulo = Triangulo(4, 6) # Instancia de Triangulo con base 4 y altura 6
print("Área del triángulo:", triangulo.calcular_area()) # Imprime el area del
triangulo
```

V. Crea dos implementaciones de la siguiente clase abstracta:

```
sS  from abc import ABCMeta, abstractmethod # Se importan algunas librerias
from abc import ABCMeta, abstractmethod
class Transporte:
    __metaclass___= ABCMeta

    def __init__(self, medio):
        self.medio = medio
    #Usar el atributo "medio" para definir como
avanza
    @abstractmethod
    def avanzar(self, frase): pass

    def giraIzquierda(self):
        print "Gira a la izquierda"

    def giraDerecha(self):
        print "Gira a la derecha"

    #De acuerdo al "medio" especificar que hace para
frenar
    @abstractmethod
    def detener(self): pass
```

Solucion:

```
from abc import ABCMeta, abstractmethod # Se importan algunas librerias
class Transporte:       # Clase base Transporte
 metaclass  = ABCMeta   # Define la metaclase para Transporte
def  init (self, medio): self.medio = medio        # Constructor de la clase Transporte
#Usar el atributo "medio" para definir como avanza
@abstractmethod
def avanzar(self, frase): pass   # Metodo abstracto avanzar
def giraIzquierda(self): # Metodo giraIzquierda
   print("Gira a la izquierda")
def giraDerecha(self):  # Metodo giraDerecha
   print("Gira a la derecha")

#De acuerdo al "medio" especificar que hace para frenar
@abstractmethod
def detener(self): pass       # Clase hija Bicicleta que hereda de Transporte
class Bicicleta(Transporte): # Clase hija Bicicleta que hereda de Transporte
 def  init (self): # Constructor de la clase Bicicleta
   super(). init ("terrestre") # Llama al constructor de la clase padre con medio "terrestre"
 def avanzar(self, frase): # Metodo para avanzar la bicicleta
   print (f"La bicicleta avanza pedaleando: {frase}")# Mensaje de avance de la bicicleta
 def detener(self): #    Metodo para detener la bicicleta
   print ("La bicicleta frena con los frenos de mano") # Mensaje de detencion de la bicicleta
class Barco(Transporte): # Clase hija Barco que hereda de Transporte
 def  init (self): # Constructor de la clase Barco
   super(). init ("acuático") # Llama al constructor de la clase padre con medio "acuático"
 def avanzar(self, frase): # Metodo para avanzar el barco
   print (f"El barco avanza navegando: {frase}") # Mensaje de avance del barco
 def detener(self): # Metodo para detener el barco
   print ("El barco frena con anclas")      # Mensaje de detencion del barco
# Crear instancias y probar métodos
bicicleta = Bicicleta()
bicicleta.avanzar("Pedaleando fuerte")
bicicleta.detener()
barco = Barco()
barco.avanzar("Remando con fuerza")
barco.detener()
```